

로봇팔의 FK, IK Matlab 결과 보고서

로봇 21기 예비단원 최윤서

문제1. 2D 3자유도 로봇팔 FK, IK 문제 해결

<코드>

```
clc; %Command Window에 이전에 출력되어있던 내용 지우는 명령어
clear; %Matlab Workspace에 저장되어있는 변수들 삭제
close all; %현재 열려있는 figure 창 모두 닫기
%% 1. 로봇팔 및 목표 위치 설정
l1 = 15; % Link 1 length
l2 = 20; % Link 2 length
l3 = 10; % Link 3 length
%외적 반지름 35인 원까지 커버 가능 / 내적 반지름 5인 반지름까지 커버 가능
target_x = 15; % 목표 EE X 위치
target_y = 30; % 목표 EE Y 위치
target = [target_x; target_y];
fprintf(' [3-DOF Robot Arm IK]\n');
fprintf('Link 1 = %.2f\n', l1);
fprintf('Link 2 = %.2f\n', l2);
fprintf('Link 3 = %.2f\n', l3);
fprintf('Target = (%.2f, %.2f)\n\n', target_x, target_y);
% 2. Algebraic IK
fprintf(' [1. Algebraic IK]\n');
% cos(theta2) 계산
theta3 = deg2rad(30);
effective_l2 = sqrt(l2^2 + l3^2 + 2*l2*l3*cos(theta3));
alpha = atan2(l3*sin(theta3), ...
    l2 + l3*cos(theta3));
cos_theta2 = (target_x^2 + target_y^2 ...
    - l1^2 - effective_l2^2) ...
    / (2*l1*effective_l2); %코사인 법칙 활용
% theta2의 두 가지 해 -> Algebraic IK는 다수의 해가 생길 수 있음
theta2_1 = acos(cos_theta2) - alpha;
theta2_2 = -acos(cos_theta2) - alpha;
% theta1 계산
theta1_1 = atan2(target_y, target_x) ...
    - atan2(effective_l2*sin(theta2_1 + alpha), ...
    l1 + effective_l2*cos(theta2_1 + alpha));
theta1_2 = atan2(target_y, target_x) ...
    - atan2(effective_l2*sin(theta2_2 + alpha), ...
    l1 + effective_l2*cos(theta2_2 + alpha));
% 두 해를 저장
algebraic_angles_1 = [theta1_1; theta2_1; theta3];
algebraic_angles_2 = [theta1_2; theta2_2; theta3];
%% 3. 대수적 IK 결과 출력
```

```

fprintf('\n[Algebraic IK - Solution 1]\n');
fprintf('theta1 = %.4f deg\n', rad2deg(theta1_1));
fprintf('theta2 = %.4f deg\n', rad2deg(theta2_1));
fprintf('theta3 = %.4f deg\n', rad2deg(theta3));
fprintf('\n[Algebraic IK - Solution 2]\n');
fprintf('theta1 = %.4f deg\n', rad2deg(theta1_2));
fprintf('theta2 = %.4f deg\n', rad2deg(theta2_2));
fprintf('theta3 = %.4f deg\n', rad2deg(theta3));
% 4. 대수적 IK 해 중 하나 선택
% 대수적 방법은 여러 해가 존재하므로
% 자신이 원하는 방향의 하나의 해를 표현
theta_alg = algebraic_angles_1; %두 개의 가능한 IK해 중에 1번해를 선택하겠다. 라는 의미
theta1_alg = theta_alg(1);
theta2_alg = theta_alg(2);
theta3_alg = theta_alg(3);
% 5. Algebraic IK -> FK 검증
[alg_x, alg_y] = forward_kinematics(theta1_alg, theta2_alg, theta3_alg, l1, l2, l3);
alg_error = norm(target - [alg_x; alg_y]);
fprintf('\n[Algebraic IK -> FK Verification]\n');
fprintf('Calculated X = %.6f\n', alg_x);
fprintf('Calculated Y = %.6f\n', alg_y);
fprintf('Position Error = %.10f\n', alg_error);
% 6. Numerical IK
fprintf('    [2. Numerical IK]\n');
% 초기 관절각
theta = deg2rad([10; 10; 10]);
% 반복 계산 설정
max_iterations = 1000;
tolerance = 1e-8;
for i = 1:max_iterations
    % 현재 관절각에서 FK 계산
    [current_x, current_y] = forward_kinematics(theta(1), theta(2), theta(3), l1, l2, l3);
    current_position = [current_x; current_y];
    % 목표 위치와 현재 위치의 오차
    error = target - current_position;
    % 오차 크기
    error_norm = norm(error);
    % 오차가 충분히 작으면 종료
    if error_norm < tolerance
        break;
    end
    % Jacobian 계산
    J = jacobian_2d(theta(1), theta(2), theta(3), l1, l2, l3);
    % Singularity 예외처리
    if rank(J) < 2
        fprintf('Singularity detected. Numerical IK stopped.\n');
        break;
    end
end

```

```

    delta_theta = pinv(J) * error;
    delta_theta = delta_theta(:);
    theta = theta + delta_theta;
end
%% 7. Numerical IK 결과 출력
theta1_num = theta(1);
theta2_num = theta(2);
theta3_num = theta(3);
fprintf('\n[Numerical IK Result]\n');
fprintf('theta1 = %.4f deg\n', rad2deg(theta1_num));
fprintf('theta2 = %.4f deg\n', rad2deg(theta2_num));
fprintf('theta3 = %.4f deg\n', rad2deg(theta3_num));
fprintf('Iteration = %d\n', i);
fprintf('Final Error = %.10e\n', error_norm);
% 8. Numerical IK -> FK 검증
[num_x, num_y] = forward_kinematics(theta1_num, theta2_num, theta3_num, l1, l2, l3);
num_error = norm(target - [num_x; num_y]);
fprintf('\n[Numerical IK -> FK Verification]\n');
fprintf('Calculated X = %.6f\n', num_x);
fprintf('Calculated Y = %.6f\n', num_y);
fprintf('Position Error = %.10f\n', num_error);
% 9. 결과 비교
fprintf('          [Result Summary]\n');
fprintf('\n[Algebraic IK]\n');
fprintf('theta1 = %.4f deg\n', rad2deg(theta1_alg));
fprintf('theta2 = %.4f deg\n', rad2deg(theta2_alg));
fprintf('theta3 = %.4f deg\n', rad2deg(theta3_alg));
fprintf('X = %.6f\n', alg_x);
fprintf('Y = %.6f\n', alg_y);
fprintf('Error = %.10f\n', alg_error);
fprintf('\n[Numerical IK]\n');
fprintf('theta1 = %.4f deg\n', rad2deg(theta1_num));
fprintf('theta2 = %.4f deg\n', rad2deg(theta2_num));
fprintf('theta3 = %.4f deg\n', rad2deg(theta3_num));
fprintf('X = %.6f\n', num_x);
fprintf('Y = %.6f\n', num_y);
fprintf('Error = %.10f\n', num_error);
% 10. 로봇팔 시각화
figure;
hold on;
grid on;
axis equal;
% 목표점
plot(target_x, target_y, 'rx', 'MarkerSize', 12, 'LineWidth', 2);
% Algebraic IK 로봇팔
x0 = 0;
y0 = 0;
x1_alg = l1*cos(theta1_alg);

```

```

y1_alg = l1*sin(theta1_alg);
x2_alg = x1_alg + l2*cos(theta1_alg + theta2_alg);
y2_alg = y1_alg + l2*sin(theta1_alg + theta2_alg);
x3_alg = x2_alg + l3*cos(theta1_alg + theta2_alg + theta3_alg);
y3_alg = y2_alg + l3*sin(theta1_alg + theta2_alg + theta3_alg);
plot([x0 x1_alg x2_alg x3_alg], [y0 y1_alg y2_alg y3_alg], 'o--', 'LineWidth', 2);
% Numerical IK 로봇팔
x1_num = l1*cos(theta1_num);
y1_num = l1*sin(theta1_num);
x2_num = x1_num + l2*cos(theta1_num + theta2_num);
y2_num = y1_num + l2*sin(theta1_num + theta2_num);
x3_num = x2_num + l3*cos(theta1_num + theta2_num + theta3_num);
y3_num = y2_num + l3*sin(theta1_num + theta2_num + theta3_num);
plot([x0 x1_num x2_num x3_num], [y0 y1_num y2_num y3_num], 's-', 'LineWidth', 2);
xlabel('X');
ylabel('Y');
title('3-DOF Robot Arm: Algebraic IK vs Numerical IK');
legend('Target', 'Algebraic IK', 'Numerical IK', 'Location', 'best');
xlim([-50 50]);
ylim([-50 50]);
hold off;
% 11. Forward Kinematics Function
function [x, y] = forward_kinematics(theta1, theta2, theta3, l1, l2, l3)
    x = l1*cos(theta1) + l2*cos(theta1 + theta2) + l3*cos(theta1 + theta2 + theta3);
    y = l1*sin(theta1) + l2*sin(theta1 + theta2) + l3*sin(theta1 + theta2 + theta3);
end
% 12. Jacobian Function
function J = jacobian_2d(theta1, theta2, theta3, l1, l2, l3)
    J11 = -l1*sin(theta1) - l2*sin(theta1 + theta2) - l3*sin(theta1 + theta2 + theta3);
    J12 = -l2*sin(theta1 + theta2) - l3*sin(theta1 + theta2 + theta3);
    J13 = -l3*sin(theta1 + theta2 + theta3);
    J21 = l1*cos(theta1) + l2*cos(theta1 + theta2) + l3*cos(theta1 + theta2 + theta3);
    J22 = l2*cos(theta1 + theta2) + l3*cos(theta1 + theta2 + theta3);
    J23 = l3*cos(theta1 + theta2 + theta3);
    J = [J11, J12, J13;
        J21, J22, J23];
end

```

<실행 결과>

$l_1 = 15$ / $l_2 = 20$ / $l_3 = 10$ 이므로

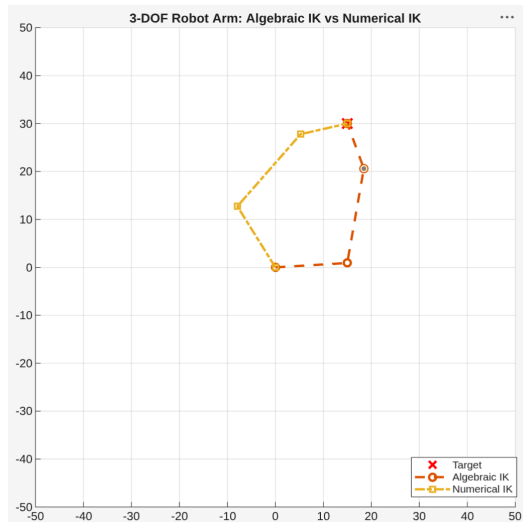
최대 도달 거리 = 45

가장 안쪽으로 접었을 때는 가장 긴 링크와 나머지 링크들의 합의 차 = $20 - (15+10)$ 인데 음수이므로

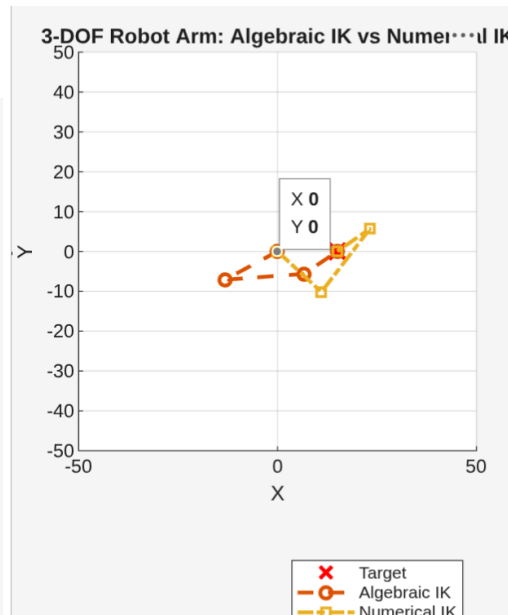
최소 도달거리 = 0

따라서 $0 \leq r \leq 45$ 범위까지 작동 가능

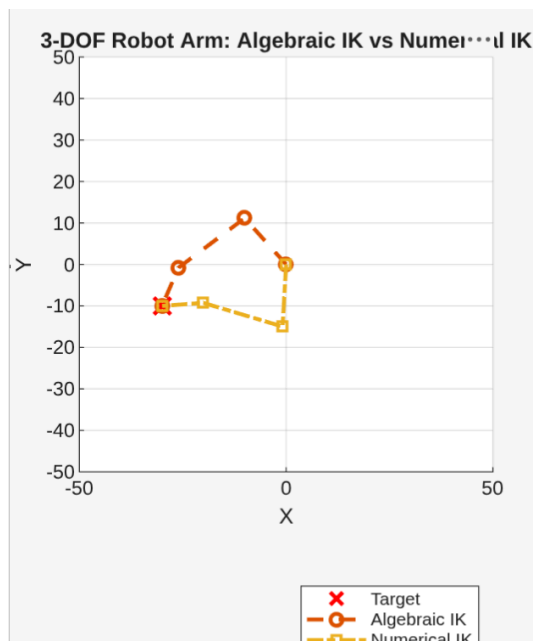
목표 위치: (15,30)



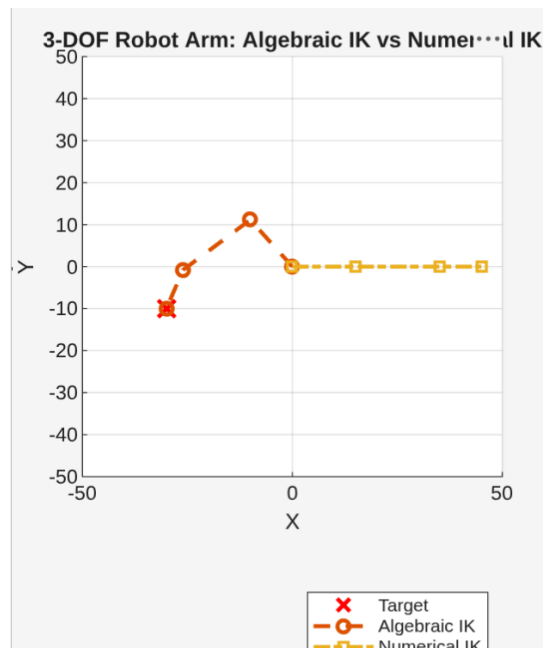
목표 위치 (15,0)



목표 위치 (-30, -10)



일부러 $\theta = \text{deg2rad}([0; 0; 0])$ 으로 바뀌어서 singularity 오류 발생시켰을 때



문제2. 3D 2자유도 로봇팔 FK, IK 문제 해결

<코드>

```
clc; %Command Window에 이전에 출력되어있던 내용 지우는 명령어
clear; %Matlab Workspace에 저장되어있는 변수들 삭제
close all; %현재 열려있는 figure 창 모두 닫기
%% 1. 로봇팔 및 목표 위치 설정
L1 = 15; % Link 1 length
L2 = 20; % Link 2 length
target_x = 10; % 목표 EE X 위치
target_y = 8; % 목표 EE Y 위치
target_z = 15; % 목표 EE Z 위치
target = [target_x; target_y; target_z];
fprintf(' [2-DOF Robot Arm IK]\n');
fprintf('Link 1 = %.2f\n', L1);
fprintf('Link 2 = %.2f\n', L2);
fprintf('Target = (%.2f, %.2f, %.2f)\n\n', target_x, target_y, target_z);
% 2. Algebraic IK
fprintf(' [1. Algebraic IK]\n');
% theta1 계산
theta1_alg = atan2(target_y, target_x);
% theta2 계산
r = sqrt(target_x^2 + target_y^2);
z = target_z - L1;
theta2_alg = atan2(z, r);
% 3. 대수적 IK 결과 출력
fprintf('\n[Algebraic IK]\n');
fprintf('theta1 = %.4f deg\n', rad2deg(theta1_alg));
fprintf('theta2 = %.4f deg\n', rad2deg(theta2_alg));
% 4. Algebraic IK -> FK 검증
[alg_x, alg_y, alg_z] = forward_kinematics(theta1_alg, theta2_alg, L1, L2);
alg_error = norm(target - [alg_x; alg_y; alg_z]);
fprintf('\n[Algebraic IK -> FK Verification]\n');
fprintf('Calculated X = %.6f\n', alg_x);
fprintf('Calculated Y = %.6f\n', alg_y);
fprintf('Calculated Z = %.6f\n', alg_z);
fprintf('Position Error = %.10f\n', alg_error);
% 5. Numerical IK
fprintf('\n [2. Numerical IK]\n');
% 초기 관절각
theta = deg2rad([10; 10]);
% 반복 계산 설정
max_iterations = 1000;
tolerance = 1e-8;
for i = 1:max_iterations
    % 현재 관절각에서 FK 계산
    [current_x, current_y, current_z] = forward_kinematics( ...
```

```

    theta(1), theta(2), L1, L2);
current_position = [current_x; current_y; current_z];
% 목표 위치와 현재 위치의 오차
error = target - current_position;
% 오차 크기
error_norm = norm(error);
% 오차가 충분히 작으면 종료
if error_norm < tolerance
    break;
end
% Jacobian 계산
J = jacobian_3d(theta(1), theta(2), L2);
% Singularity 예외처리
if rank(J) < 2
    fprintf('Singularity detected. Numerical IK stopped.\n');
    break;
end
delta_theta = pinv(J) * error;
delta_theta = delta_theta(:);
theta = theta + delta_theta;
end
% 6. Numerical IK 결과 출력
theta1_num = theta(1);
theta2_num = theta(2);
fprintf('\n[Numerical IK Result]\n');
fprintf('theta1 = %.4f deg\n', rad2deg(theta1_num));
fprintf('theta2 = %.4f deg\n', rad2deg(theta2_num));
fprintf('Iteration = %d\n', i);
fprintf('Final Error = %.10e\n', error_norm);
% 7. Numerical IK -> FK 검증
[num_x, num_y, num_z] = forward_kinematics(theta1_num, theta2_num, L1, L2);
num_error = norm(target - [num_x; num_y; num_z]);
fprintf('\n[Numerical IK -> FK Verification]\n');
fprintf('Calculated X = %.6f\n', num_x);
fprintf('Calculated Y = %.6f\n', num_y);
fprintf('Calculated Z = %.6f\n', num_z);
fprintf('Position Error = %.10f\n', num_error);
% 8. 결과 비교
fprintf('\n          [Result Summary]\n');
fprintf('\n[Algebraic IK]\n');
fprintf('theta1 = %.4f deg\n', rad2deg(theta1_alg));
fprintf('theta2 = %.4f deg\n', rad2deg(theta2_alg));
fprintf('X = %.6f\n', alg_x);
fprintf('Y = %.6f\n', alg_y);
fprintf('Z = %.6f\n', alg_z);
fprintf('Error = %.10f\n', alg_error);
fprintf('\n[Numerical IK]\n');
fprintf('theta1 = %.4f deg\n', rad2deg(theta1_num));

```

```

fprintf('theta2 = %.4f deg\n', rad2deg(theta2_num));
fprintf('X = %.6f\n', num_x);
fprintf('Y = %.6f\n', num_y);
fprintf('Z = %.6f\n', num_z);
fprintf('Error = %.10f\n', num_error);
% 9. 로봇팔 시각화
figure;
hold on;
grid on;
axis equal;
% 목표점
plot3(target_x, target_y, target_z, ...
    'rx', 'MarkerSize', 12, 'LineWidth', 2);
% Algebraic IK 로봇팔
x0_alg = 0;
y0_alg = 0;
z0_alg = 0;
x1_alg = 0;
y1_alg = 0;
z1_alg = L1;
x2_alg = alg_x;
y2_alg = alg_y;
z2_alg = alg_z;
plot3([x0_alg x1_alg x2_alg], [y0_alg y1_alg y2_alg], [z0_alg z1_alg z2_alg], 'o--',
    'LineWidth', 2);
% Numerical IK 로봇팔
x0_num = 0;
y0_num = 0;
z0_num = 0;
x1_num = 0;
y1_num = 0;
z1_num = L1;
x2_num = num_x;
y2_num = num_y;
z2_num = num_z;
plot3([x0_num x1_num x2_num], [y0_num y1_num y2_num], [z0_num z1_num
z2_num], 's-.', 'LineWidth', 2);
xlabel('X');
ylabel('Y');
zlabel('Z');
title('2-DOF Robot Arm: Algebraic IK vs Numerical IK');
legend('Target', 'Algebraic IK', 'Numerical IK', ...
    'Location', 'best');
xlim([-30 30]);
ylim([-30 30]);
zlim([0 40]);
view(3);
hold off;

```

% 10. Forward Kinematics Function

```
function [x, y, z] = forward_kinematics(theta1, theta2, L1, L2)
    x = L2*cos(theta2)*cos(theta1);
    y = L2*cos(theta2)*sin(theta1);
    z = L1 + L2*sin(theta2);
```

end

% 11. Jacobian Function

```
function J = jacobian_3d(theta1, theta2, L2)
    J11 = -L2*cos(theta2)*sin(theta1);
    J12 = -L2*sin(theta2)*cos(theta1);
    J21 = L2*cos(theta2)*cos(theta1);
    J22 = -L2*sin(theta2)*sin(theta1);
    J31 = 0;
    J32 = L2*cos(theta2);
    J = [J11, J12;
        J21, J22;
        J31, J32];
```

end

<실행 화면>

$l_1 = 15, l_2 = 20$ 이므로

작업 공간은 중심이 (0,0,15)인 반지름이 20인 구이다.

x: 약 -20~20

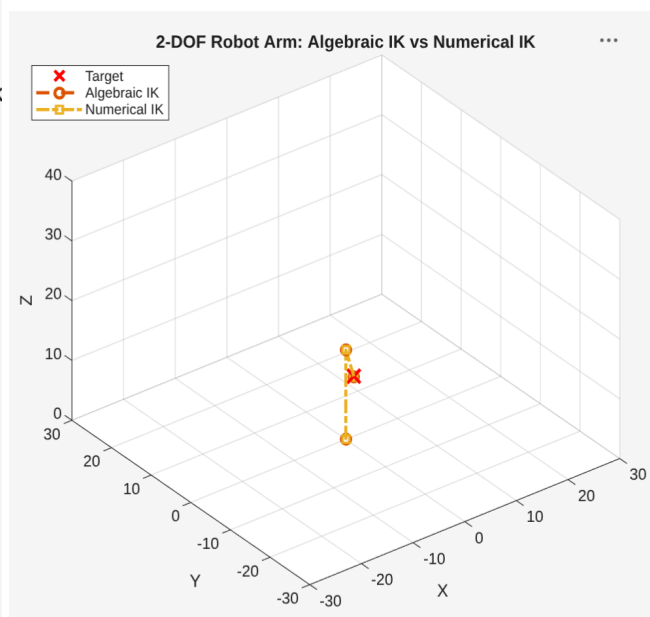
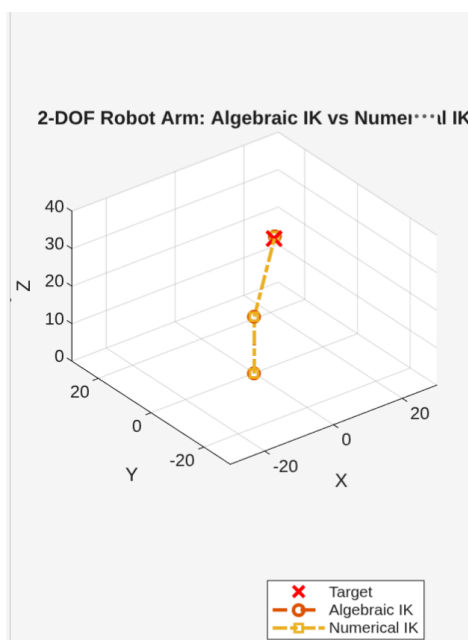
y: 약 -20~20

z: -5~35

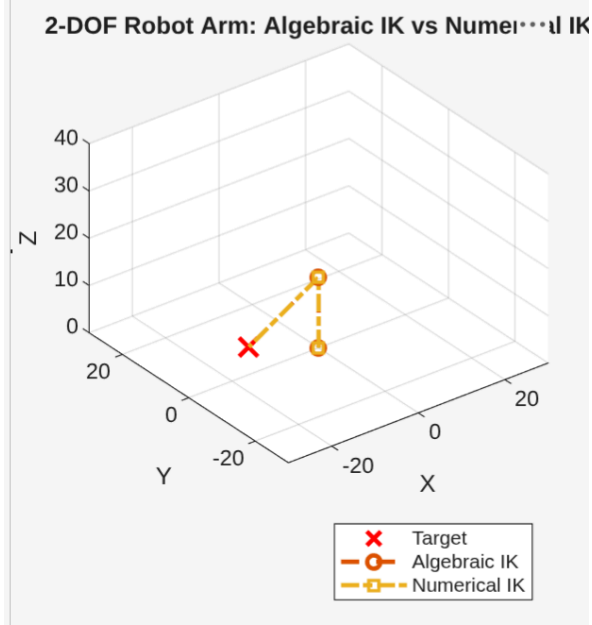
범위 내에서 작동 가능

목표 위치 (12,8,28)

목표 위치(-10, 11, 2)



목표위치 (-10, 8, 0)



목표 위치치 (-15, 14, 3)

